to handle the base class entity and so, if someone tries to assign a bigger (yes, the derived object is usually bigger than the base class object) object to it, then obviously, the extra things will be deprecated or truncated. While the rationale behind object slicing seems obvious and easy to understand, its implications can be surprising.

It can be very tricky, if used unknowingly, especially when passing objects to functions, returning values from functions, assigning one object to another or creating (or copying) objects from another object. One should be very careful in such instances, during which you are bound to encounter object slicing.

Now let's look at one scenario which can throw up some surprises.

Let's modify the base class to have member 'a_' declared as protected (so that it is accessible by the derived class) and let's also modify the *Derived::print()* method to also display the base class member 'a_'.

```
#include <iostream>
using namespace std;

class Base
{
public:
Base(int a)
    {
      a_ = a;
    }

void print()
    {
cout<< "In Base: a_ = " << a_ <<endl;
    }

protected:
int a_;    // Declared as protected.
};


class Derived : public Base
{
public:
Derived(int a, int b):Base(a)
    {
      b_ = b;
    }

void print()
    {
cout<< "In Derived: a_ = " << a_ << " and b_ = " << b_
<<endl;
      // It can access a_ as it was declared protected.
    }
private:
int b_;
};


int main()
{
  Base b(10);
  Derived d(11, 21), dd(15, 25);

  b.print();
  d.print();

  Base &bb = d;   // take it by reference object of Base

  bb.print();   // Since print() is not defined 'virtual', it
would call
  // Base::print() as bb is declared as Base reference and
the
  // static binding will force it that way.

  bb = dd;      // troublesome assignment!
  bb.print();

  return 0;
}
```

The output wouldn't be much different than what we would assume.

```
------ output ------

In Base: a_ = 10
In Derived: a_ = 11 and b_ = 21
In Base: a_ = 11      <-- Object d got sliced
In Base: a_ = 15      <-- Object dd got sliced
```

Now, let's declare *Base::print()* as virtual.

```
class Base
{
  …
virtual void print()
    {
cout<< "In Base: a_ = " << a_ <<endl;
    }
  …
}
```

Let's run the same main program again. This time, the *bb.print()* call will invoke *Derived::print()* as the *print()* method is declared virtual in base. Now, notice the output carefully: